**UNIVERSITAS INDONESIA**

# LEX2KG: AUTOMASI PEMBUATAN KNOWLEDGE GRAPH DARI DOKUMEN PERATURAN PERUNDANG-UNDANGAN INDONESIA

**SKRIPSI**

**MUHAMAD ABDURAHMAN**
**1706040095**

**FAKULTAS ILMU KOMPUTER**
**PROGRAM STUDI ILMU KOMPUTER**
**DEPOK**
**JULI 2021**

**UNIVERSITAS INDONESIA**

# LEX2KG: AUTOMASI PEMBUATAN KNOWLEDGE GRAPH DARI DOKUMEN PERATURAN PERUNDANG-UNDANGAN INDONESIA

**SKRIPSI**

**Diajukan sebagai salah satu syarat untuk memperoleh gelar**
**Sarjana Ilmu Komputer**

**MUHAMAD ABDURAHMAN**
**1706040095**

**FAKULTAS ILMU KOMPUTER**
**PROGRAM STUDI ILMU KOMPUTER**
**DEPOK**
**JULI 2021**

# HALAMAN PERSETUJUAN

| | | |
|---|---|---|
| **Judul** | : | Lex2KG: Automasi Pembuatan Knowledge Graph dari Dokumen Peraturan Perundang-undangan Indonesia |
| **Nama** | : | Muhamad Abdurahman |
| **NPM** | : | 1706040095 |

Laporan Skripsi ini telah diperiksa dan disetujui.

Tanggal Bulan Tahun

Fariz Darari, Ph.D.

Pembimbing Skripsi

# HALAMAN PERNYATAAN ORISINALITAS

**Skripsi ini adalah hasil karya saya sendiri,**
**dan semua sumber baik yang dikutip maupun dirujuk**
**telah saya nyatakan dengan benar.**

| | | |
|---|---|---|
| **Nama** | : | **Muhamad Abdurahman** |
| **NPM** | : | **1706040095** |
| **Tanda Tangan** | : | |
| | | |
| **Tanggal** | : | **Tanggal Bulan Tahun** |

# HALAMAN PENGESAHAN

| | | |
|---|---|---|
| Skripsi ini diajukan oleh | : | |
| Nama | : | Muhamad Abdurahman |
| NPM | : | 1706040095 |
| Program Studi | : | Ilmu Komputer |
| Judul Skripsi | : | Lex2KG: Automasi Pembuatan Knowledge Graph dari Dokumen Peraturan Perundang-undangan Indonesia |

**Telah berhasil dipertahankan di hadapan Dewan Penguji dan diterima sebagai bagian persyaratan yang diperlukan untuk memperoleh gelar Sarjana Ilmu Komputer pada Program Studi Ilmu Komputer, Fakultas Ilmu Komputer, Universitas Indonesia.**

**DEWAN PENGUJI**

| | | | | |
|---|---|---|---|---|
| Pembimbing 1 | : | Fariz Darari, Ph.D. | ( | ) |
| Penguji 1 | : | Penguji Pertama Anda | ( | ) |
| Penguji 2 | : | Penguji Kedua Anda | ( | ) |

| | | |
|---|---|---|
| Ditetapkan di | : | Depok |
| Tanggal | : | Tanggal Bulan Tahun |

# HALAMAN PERNYATAAN PERSETUJUAN PUBLIKASI TUGAS AKHIR UNTUK KEPENTINGAN AKADEMIS

Sebagai sivitas akademik Universitas Indonesia, saya yang bertanda tangan di bawah ini:

| | | |
|---|---|---|
| **Nama** | : | Muhamad Abdurahman |
| **NPM** | : | 1706040095 |
| **Program Studi** | : | Ilmu Komputer |
| **Fakultas** | : | Ilmu Komputer |
| **Jenis Karya** | : | Skripsi |

demi pengembangan ilmu pengetahuan, menyetujui untuk memberikan kepada Universitas Indonesia **Hak Bebas Royalti Noneksklusif (Non-exclusive Royalty Free Right)** atas karya ilmiah saya yang berjudul:

Lex2KG: Automasi Pembuatan Knowledge Graph dari Dokumen Peraturan Perundang-undangan Indonesia

beserta perangkat yang ada (jika diperlukan). Dengan Hak Bebas Royalti Noneksklusif ini Universitas Indonesia berhak menyimpan, mengalihmedia/formatkan, mengelola dalam bentuk pangkalan data (database), merawat, dan memublikasikan tugas akhir saya selama tetap mencantumkan nama saya sebagai penulis/pencipta dan sebagai pemilik Hak Cipta.

Demikian pernyatan ini saya buat dengan sebenarnya.

| | | |
|---|---|---|
| Dibuat di | : | Depok |
| Pada tanggal | : | Tanggal Bulan Tahun |

Yang menyatakan

(Muhamad Abdurahman)

# KATA PENGANTAR

Lorem Ipsum

Depok, Tanggal Bulan Tahun

Muhamad Abdurahman

# ABSTRAK


| | | |
|---|---|---|
| Nama | : | Muhamad Abdurahman |
| Program Studi | : | Ilmu Komputer |
| Judul | : | Lex2KG: Automasi Pembuatan Knowledge Graph dari Dokumen Peraturan Perundang-undangan Indonesia |



Peraturan perundang-undangan pada umumnya tersedia dalam bentuk berkas PDF yang bersifat tidak *machine-readable*. *Knowledge graph* adalah struktur data berbentuk *graph* yang menggambarkan entitas dunia nyata beserta keterkaitannya. Pada penelitian ini berfokus pada pengembangan *framework* untuk mengonversi peraturan perundang-undangan di Indonesia dari dokumen PDF menjadi *knowledge graph* dan memberikan contoh *use case* dari *knowledge graph* tersebut. *Knowledge graph* peraturan perundang-undangan tersebut mengandung data-data berupa metadata peraturan, struktur peraturan, teks peraturan, serta relasi antara peraturan seperti amandemen dan rujukan. Dari 1669 dokumen PDF yang diunduh dari situs Jaringan Dokumentasi dan Informasi Hukum DPR RI, 784 dokumen diantaranya berhasil dikonversi menjadi *knowledge graph* dengan total 1.159.231 *triple*. Salah satu peraturan yang berhasil dikonversi adalah UU 11/2020 tentang Cipta Kerja yang kontennya bersifat relatif kompleks dan berukuran besar. Penelitian ini juga mendemonstrasikan pemanfaatan *knowledge graph* tersebut untuk *chat bot*, *visualisasi*, dan *query*.



**Kata kunci:**
*Knowledge Graph*, Peraturan perundang-undangan, SPARQL, *Resource Definition Framework*

# ABSTRACT

Name : Muhamad Abdurahman
Program : Ilmu Komputer
Title : Lex2KG: Automated Legal KG Population from Indonesian Legal Documents

//TODO: Ini ntar aja klo yg abstrak bhs indo udah final

**Key words:**
Keyword one, keyword two

# DAFTAR ISI

**HALAMAN JUDUL** **i**

**LEMBAR PERSETUJUAN** **ii**

**LEMBAR PERNYATAAN ORISINALITAS** **iii**

**LEMBAR PENGESAHAN** **iv**

**LEMBAR PERSETUJUAN PUBLIKASI ILMIAH** **v**

**KATA PENGANTAR** **vi**

**ABSTRAK** **vii**

**Daftar Isi** **ix**

**Daftar Gambar** **xi**

**Daftar Tabel** **xii**

**Daftar Kode Program** **xiii**

**Daftar Lampiran** **xiii**

**1 PENDAHULUAN** **1**
1.1 Latar Belakang . . . . . 1
1.2 Permasalahan . . . . . 5
1.3 Batasan Permasalahan . . . . . 5
1.4 Sistematika Penulisan . . . . . 5
1.5 Metodologi Penelitian . . . . . 5

**2 TINJAUAN PUSTAKA** **7**
2.1 Knowledge Graph . . . . . 7
2.2 Resource Description Framework . . . . . 8
2.3 SPARQL . . . . . 9
2.3.1 Contoh Query 1 . . . . . 9
2.3.2 Contoh Query 2 . . . . . 10
2.3.3 Contoh Query 3 . . . . . 10
2.4 Peraturan Perundang-undangan . . . . . 11
2.5 Pemodelan European Legislation Identifier (ELI) . . . . . 12

**3 PERANCANGAN** **13**
3.1 Perancangan URI . . . . . 13

3.2 Perancangan URI Class . . . . . 13
3.3 Perancangan URI Komponen Peraturan Perundang-undangan . . . . . 14
3.4 Perancangan URI Properti . . . . . 16

**4 IMPLEMENTASI** **18**
4.1 OCR Ulang Berkas PDF . . . . . 18
4.2 Ekstraksi Berkas PDF menjadi Data *Span* . . . . . 19
4.3 Membersihkan *Noise* dari Halaman . . . . . 21
4.4 Penggabungan Data Berkas PDF Asli dan Hasil OCR Ulang . . . . . 21
4.5 Klasifikasi Dokumen . . . . . 22
4.6 Pengelompokan *span* menjadi Komponen . . . . . 23
4.6.1 Komponen Terurut . . . . . 24
4.6.2 Komponen Amandemen . . . . . 25
4.7 Deteksi Sitasi . . . . . 26
4.8 Visualisasi dengan Markdown . . . . . 28
4.9 Data to triple . . . . . 29
4.10 triple to ttl . . . . . 29
4.11 Query . . . . . 29

**5 EVALUASI DAN ANALISIS HASIL** **30**
5.1 Aplikasi *Knowledge Graph* Peraturan Perundang-undangan . . . . . 30
5.2 Evaluasi Hasil Konversi . . . . . 30
5.3 Evaluasi Manual . . . . . 30
5.4 Evaluasi Otomatis . . . . . 30

**6 Penutup** **31**
6.1 Kesimpulan . . . . . 31
6.2 Saran . . . . . 31

**Daftar Referensi** **32**

# DAFTAR GAMBAR


1.1 Cuplikan ekstraksi data terstruktur dan data tidak terstruktur dari dokumen PDF UU 11/2020 tentang Cipta Kerja . . . . . 2
1.2 Bagan Alir Pembuatan dan Query KG Peraturan Perundang-undangan . . 3

2.1 Contoh *Knowledge Graph* . . . . . 7
2.2 Contoh Triple . . . . . 8

4.1 Gambar dokumen PDF yang akan diekstraksi . . . . . 20
4.2 Contoh header . . . . . 21
4.3 Contoh nomor tidak terpindai . . . . . 21
4.4 Ilustrasi Ekstraksi Daftar Span menjadi Data Struktur Komponen . . . . . 24
4.5 Contoh Amendemen pada UU Nomor 11 Tahun 2020 . . . . . 25
4.6 Deteksi Komponen Amandemen . . . . . 26
4.7 Perbandingan visualisasi dengan yaml (kiri) dan markdown (kanan) . . . 28

# DAFTAR TABEL

3.1 Long table caption. . . . . 14
3.2 Long table caption. . . . . 15
3.3 Long table caption. . . . . 16
3.4 Long table caption. . . . . 17

4.1 Long table caption. . . . . 27

# DAFTAR KODE PROGRAM

1.1 Contoh KG tentang metadata untuk suatu dokumen legal dengan *vocabulary* Schema.org . . . . . 4
2.1 Example from external file . . . . . 10

# DAFTAR LAMPIRAN

Lampiran 1. CHANGELOG . . . . . 34

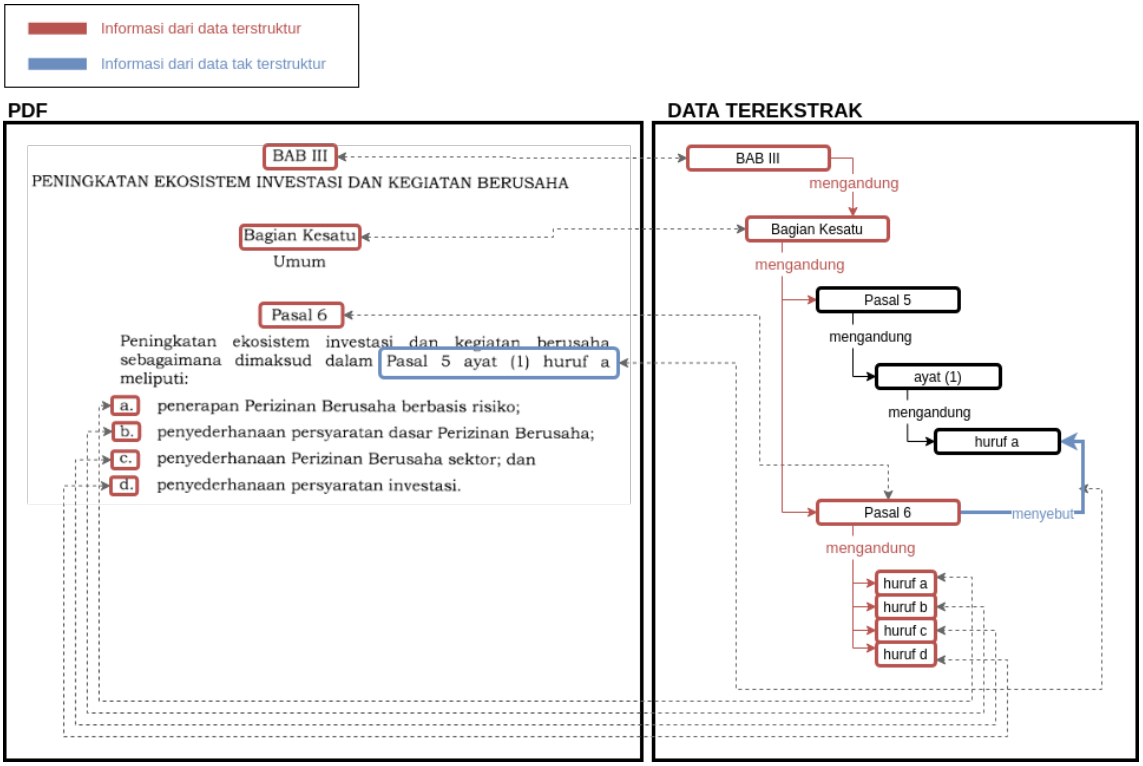
# BAB 1
# PENDAHULUAN

## 1.1 Latar Belakang

Peraturan perundang-undangan adalah peraturan tertulis yang memuat norma hukum yang mengikat secara umum dan dibentuk atau ditetapkan oleh lembaga negara atau pejabat yang berwenang melalui prosedur yang ditetapkan dalam peraturan perundang-undangan, sesuai yang dijelaskan dalam Undang-Undang Republik Indonesia Nomor 12 Tahun 2011 (selanjutnya disebut dengan UU 12/2011). Peraturan perundang-undangan dapat digunakan untuk menjawab pertanyaan-pertanyaan yang berkaitan dengan hukum, seperti:

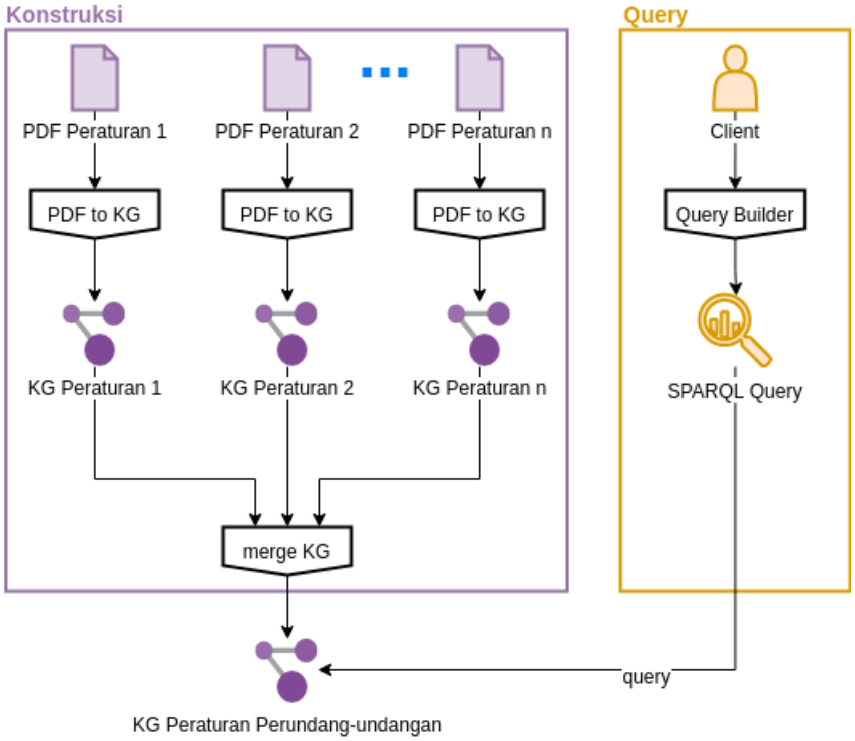
- Peraturan apa yang berlaku pada daerah X yang disahkan di tahun Y?
- Apa relasi suatu peraturan dengan peraturan lain?
- Mana saja peraturan yang membahas tentang topik W atau X tapi bukan topik Y dan bukan Z?
- Apa isi dari Pasal X ayat (Y) huruf Z dari suatu peraturan perundang-undangan?
- Apa saja bab-bab yang terdapat pada peraturan X?

Pertanyaan-pertanyaan tersebut umumnya biasa dijawab oleh seorang ahli hukum, yang hanya dapat dilakukan dalam skala kecil dan biaya relatif mahal. Komputer dapat menjadi alternatif untuk aplikasi tersebut dalam skala lebih besar dan biaya lebih murah, jika peraturan perundang-undangan berupa data terstruktur yang dapat diolah oleh komputer. Akan tetapi, data peraturan perundang-undangan umumnya dibuat dalam bentuk semi-terstruktur, yaitu berupa dokumen PDF dengan struktur konten seperti aturan penomoran dan aturan pembagian (*sectioning*), sementara penulisan isi peraturannya sendiri dalam bentuk teks bahasa manusia (natural language) yang tak terstruktur (unstructured).




**Gambar 1.1:** Cuplikan ekstraksi data terstruktur dan data tidak terstruktur dari dokumen PDF UU 11/2020 tentang Cipta Kerja

Gambar 1.1 menampilkan contoh ekstraksi informasi dari UU 11/2020 tentang Cipta Kerja. Pada gambar tersebut terlihat bahwa terdapat dua jenis ekstraksi data yaitu ekstraksi dari data terstruktur (merah) dan ekstraksi dari data tidak terstruktur (biru). Contoh data terstruktur pada dokumen tersebut adalah *section* pada dokumen seperti **Bab III**, **Bagian Kesatu**, **Pasal 6**, dan **huruf a - d**. Contoh data tidak terstruktur pada dokumen tersebut adalah semua teks yang terdapat pada *section* tersebut seperti teks "penerapan Perizinan Berusaha berbasis risiko;" yang terdapat pada **huruf a**. Manusia perlu membaca manual untuk mengetahui bahwa UU 11/2020 mengandung **Bab III**, **Bab III** mengandung **Bagian Kesatu**, **Bagian Kesatu** mengandung **Pasal 6**, dan **Pasal 6** mengandung **huruf a - d**. Pada gambar tersebut, dalam mengekstraksi informasi "UU 11/2020 Pasal 6 merujuk kepada Pasal 5 ayat (1) huruf a", manusia pertama-tama perlu membaca dari awal teks pasal-pasal pada UU perujukan (citation), dan kemudian meng-*infer* informasi lengkap berdasarkan konteks peraturan perundang-undangan.




**Gambar 1.2:** Bagan Alir Pembuatan dan Query KG Peraturan Perundang-undangan

*Knowledge graph* (KG) adalah struktur data yang (i) menggambarkan entitas dunia nyata dan keterkaitannya, tersusun dalam bentuk *graph*, (ii) mendefinisikan kelas entitas dan relasi antar entitas dalam bentuk skema, (iii) membolehkan sembarang entitas memiliki relasi satu sama lain dan (iv) mencakup berbagai topik domain (Paulheim, 2016). Dengan melakukan ekstraksi data dari dokumen yang bersifat semi-terstruktur, peraturan perundang-undangan dapat diubah menjadi *knowledge graph*. Sebagai manfaatnya, dapat fitur-fitur berbasis *query* seperti *question answering*, *reasoning* dan visualisasi data. Gambar 1.2 menampilkan proses konstruksi *knowledge graph* dari beberapa berkas PDF peraturan perundang-undangan dan *query* pada KG tersebut. Pada skripsi ini, akan dilakukan konversi dokumen peraturan perundang-undangan menjadi data terstruktur dalam bentuk *knowledge graph*, dan memberikan contoh aplikasi dari *knowledge graph* peraturan perundang-undangan tersebut. Beberapa usaha telah dilakukan untuk membuat *vocabulary* untuk *knowledge graph* peraturan perundang-undangan seperti ELI [1], Schema.org [2], dan Wikidata [3].

[1] `https://eur-lex.europa.eu/eli-register/about.html`
[2] `https://schema.org/Legislation`
[3] `https://www.wikidata.org/wiki/Wikidata:WikiProject_Law`

```
@prefix schema: <http://schema.org/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

<http://data.europa.eu/eli/dir/1980/181/oj>
  a schema:Legislation ;
  schema:about <http://eurovoc.europa.eu/1896> ;
  schema:isBasedOn <http://njh.me/11957E100.html> ;
  schema:legislationDate "1979-12-20"^^schema:Date ;
  schema:legislationPassedBy "Council of the European Union"^^xsd:string ;
  schema:name "Council Directive 80/181/EEC of 20 December 1979 on the approximation
  ↪ of the laws of the Member States relating to units of measurement and on the
  ↪ repeal of Directive 71/354/EEC"^^xsd:string ;

<http://eurovoc.europa.eu/1896>
  a schema:Thing ;
  schema:name "Metrology"^^xsd:string .

<http://njh.me/11957E100.html> a schema:Legislation .
```

**Kode 1.1:** Contoh KG tentang metadata untuk suatu dokumen legal dengan *vocabulary* Schema.org

Pada Kode 1.1 diberikan contoh *knowledge graph* yang dibuat dengan *vocabulary* Schema.org. Contoh KG tersebut memuat metadata peraturan seperti tentang apa peraturan tersebut (`schema:about`), apa dasar dari peraturan tersebut (`schema:isBasedOn`), dan nama dari peraturan tersebut (`schema:name`). Akan tetapi, baik Schema.org maupun vocabulary-vocabulary lainnya seperti pada ELI dan Wikidata, tidak dapat merepresentasikan struktur beserta isi peraturan. Selain itu, *vocabulary* tersebut dirancang secara generik, sehingga tidak dapat merepresentasikan data yang secara spesifik terdapat pada peraturan perundang-undangan di Indonesia. Sehingga dengan membuat *vocabulary* untuk struktur data dan peraturan perundang-undangan di Indonesia, diharapkan dapat menjadi kontribusi dalam mendukung pembuatan *knowledge graph* untuk dokumen peraturan perundang-undangan.

Lex2KG merupakan *framework* yang dikembangkan pada penelitian ini, dapat mengonversi dokumen menjadi *knowledge graph* secara otomatis. Belum ada pekerjaan terkait yang melakukan automasi konversi dokumen peraturan perundang-undangan menjadi *knowledge graph* secara otomatis. Oleh karena itu Lex2KG diharapkan dapat menjadi kontribusi yang mendukung pembuatan dan penggunaan *knowledge graph*.

## 1.2 Permasalahan

Berikut ini adalah rumusan permasalahan dari penelitian yang dilakukan:

- Bagaimana cara membuat *legal ontology* untuk memperjelas *requirement* dan mempermudah konversi dokumen PDF menjadi *knowledge graph*?
- Bagaimana konversi dokumen peraturan perundang-undangan menjadi *knowledge graph* dapat dilakukan secara otomatis?
- Apa saja contoh aplikasi dari knowledge graph peraturan perundang-undangan tersebut?

// TODO: bagaimana melakukan proof of concept untuk yang udah dibuat, detail untuk maintain, general large scale, evaluasi konversi

// TODO: bolongnya State of the art adalah ...

// TODO: 1. Bagaimana merancang suatu sistem untuk melakukan automated population?

2. memperjelas requirement dan menangkap informasi yang terkandung di dalam dokumen legal yang ada, sehingga mendukung konversi pdf menjadi knowledge graph.

3. bagaimana evaluasi use case terhadap kg peraturan perundang-undangan tersebut serta evaluasi konversi / populasi dalam skala besar

## 1.3 Batasan Permasalahan

// TODO: fokus yang mainteined (UU + non UU), dan coba parsing UU hasil scraping

// TODO: batasan: tipe query/question nya belum yang super complex (contoh: jika saya usia 12 tahun, apakah boleh kerja menurut UU ketenaga kerjaan?)

## 1.4 Sistematika Penulisan

## 1.5 Metodologi Penelitian

// TODO: Maintained docs masukin sini

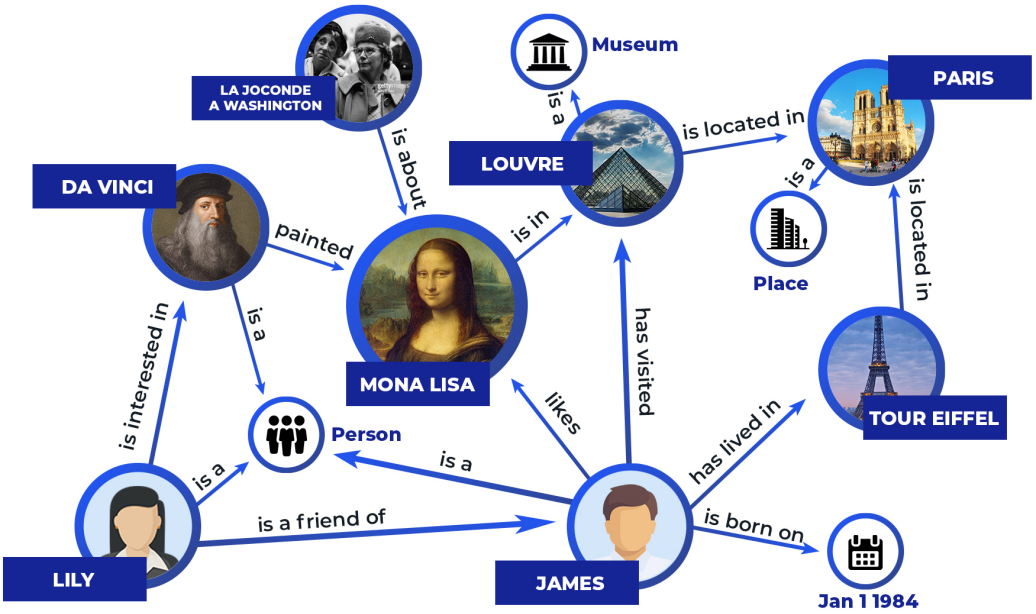
// TODO: Utk yg metodologi, sbnrnya standar aja, dimulai dari pertanyaan riset, lalu pengembangan ontology (URI scheming masuk ke pengembagan ontology), pengembangan sistem konversi, baru lalu use case evaluation (ini yg legal KG advanced querying, legal KG chatbot, legal KG visualization) baru large scale eval (ini yg coba konversi peraturan dlm jumlah besar dan kita sampling utk evaluasi correctnessnya)

// TODO: suatu KG construction dimulai dengan competency questions (requirements). Saran sy: coba baca serta rangkum (di Bab 2) dan terapkan ini: www.ksl.stanford.edu/people/dlm/papers/ontology-tutorial-noy-mcguinness-abstract.html???

# BAB 2
# TINJAUAN PUSTAKA

## 2.1 Knowledge Graph

*Knowledge graph* merupakan model data berbasis *graph* yang menggambarkan entitas dunia nyata dan hubungan antara entitas-entitas tersebut (Seth, 2019). Sebagai contoh, pada gambar dibawah dapat dilihat terpadat entitas-entitas yang ditandai dengan bentuk lingkaran, dan hubungan antara entitas-entitas ditandai dengan bentuk anak panah. Dapat dilihat pada gambar dibawah bahwa terdapat entitas "James" dan "Louvre" dan properti "has visited" yang mengarah dari "James" ke "Louvre". Relasi tersebut dapat kita anggap sebagai sebuah pengetahuan yang dalam bahasa manusia dapat dituliskan sebagai "James pernah mengunjungi Louvre". Model data seperti ini cocok untuk menyimpan data dengan banyak jenis relasi antara entitas seperti pengetahuan umum, dibandingkan dengan *relational database* yang cocok untuk menyimpan data dengan relasi antar entitas yang sudah diketahui dan terbatas.




**Gambar 2.1:** Contoh *Knowledge Graph*

## 2.2 Resource Description Framework

Resource Description Framework (RDF) merupkan model data yang memberikan pernyataan tentang suatu *resource* dalam bentuk subjek-properti-objek, atau sering disebut *triple*. Sebuah *knowledge graph* dapat direpresentasikan sebagai kumpulan *triple*. Subyek dan properti dalam *triple* direpresentasikan sebagai Uniform Resource Identifier (URI), sedangkan objek dapat direpresentasikan dengan URI atau *string literal*. Pada gambar dibawah, dapat dilihat terdapat *knowledge graph* dengan dua *triple*. Pada triple 1 subjek, properti, dan objek berturut-turut adalah `https://example.org/james`, `https://example.org/has_visited`, `https://example.org/louvre` yang mana semuanya berbentuk URI. Sedangkan pada triple 2 subjek, properti, dan objek berturut-turut adalah `https://example.org/james`, `https://example.org/has_name`, "james" . `https://example.org/james`, `https://example.org/louvre` berturut-turut merupakan notasi untuk *resource* entitas James dan Louvre, `https://example.org/has_visited` dan `https://example.org/has_name` merupakan properti yang menjelaskan hubungan antara subjek dan objek, dan "james" adalah *string literal* yang menunjukan nilai dari nama James.

**Gambar 2.2:** Contoh Triple

Terdapat beberapa format berkas dan sintaks untuk mengekspresikan RDF. Pada penelitian ini, penulis menggunakan format Terse RDF Triple Language (Turtle). Sebagai contoh, *knowledge graph* diatas dapat dituliskan dalam format Turtle sebagai berikut.

```
<https://example.org/james> <https://example.org/has_visited>
↪  <https://example.org/louvre> .
<https://example.org/james> <https://example.org/has_name> "james" .
```

Dengan fitur format Turtle, dua *triple* diatas dapat ditulis lebih singkat dengan

mengganti prefix URI yang sama menjadi teks yang lebih singkat, dan menggabungkan penulisan beberapa *triple* dengan subjek yang sama sehingga subjek hanya perlu dituliskan sekali saja. Sebagai contoh, karena sebagaian besar URI memiliki prefix `https://example.org/`, kita dapat menggantinya dengan *string* `ex` dengan mendeklarasikan `@prefix ex: https://example.org/ .`. Kedua *triple* juga memiliki subjek yang sama yaitu `https://example.org/james`, oleh karena itu, penulisan kedua turtle bisa digabung dengan hanya menuliskan subjek sebanyak satu kali, kemudian menuliskan properti dan objek dipisah dengan tanda titik koma . Berikut adalah contoh penulisan dua *triple* diatas dalam bentuk yang lebih singkat dan lebih mudah dibaca oleh manusia.

```
@prefix ex: <https://example.org/> .

ex:james
  ex:has_visited ex:louvre ;
  ex:has_name "james" .
```

## 2.3 SPARQL

SPARQL merupakan salahsatu RDF *query language* yaitu bahasa yang mampu mengambil atau mengubah data yang disimpan dalam format RDF. Sintaks SPARQL memiliki beberapa kesamaan dengan SQL, contohnya sebuah variabel diawali dengan tanda tanya seperti `?name`. Berikut adalah penjelasan sintaks SPARQL.

- `PREFIX`: Mendeklarasikan pemetaan prefix URI menjadi *string* yang lebih singkat. Hal ini dilakukan agar *query* lebih mudah dibaca oleh manusia seperti sintaks `@prefix` pada Turtle.
- `SELECT`: Menyatakan variabel yang akan ditampilkan sebagai output dari *query*.
- `WHERE`: Menyatakan kondisi *triple* yang akan ditampilkan pada output.

Berikut adalah contoh-contoh *query* SPARQL yang dilakukan pada contoh *knowledge graph* pada subbab sebelumnya.

### 2.3.1 Contoh Query 1

*Query* ini bertujuan untuk menampilkan semua *triple* yang terdapat pada *knowledge graph*.

Output dari *query* diatas adalah sebagai berikut.

```
PREFIX ex: <https://example.org/>
SELECT ?s
       ?p
       ?o
WHERE
  {
    ?s ?p ?o
  }
```

**Kode 2.1:** Example from external file

| ?s | ?p | ?o |
|---|---|---|
| ex:james | ex:has_visited | ex:louvre |
| ex:james | ex:has_name | "james" |

### 2.3.2 Contoh Query 2

*Query* ini bertujuan untuk menampilkan semua properti dan objek pada *triple* dengan subjek `https://example.org/james`.

```
PREFIX ex: <https://example.org/>
SELECT ?p
       ?o
WHERE
  {
    ex:james ?p ?o
  }
```

Output dari *query* tersebut adalah sebagai berikut.

| ?p | ?o |
|---|---|
| ex:has_visited | ex:louvre |
| ex:has_name | "james" |

### 2.3.3 Contoh Query 3

*Query* ini bertujuan untuk menampilkan semua objek pada *triple* dengan subjek `https://example.org/james` dan properti `https://example.org/has_visited`. Artinya, *query* ini menampilkan semua tempat yang pernah dikunjungi james.

```
PREFIX ex: <https://example.org/>
SELECT ?visited_place
WHERE
  {
    ex:james ex:has_visited ?visited_place
  }
```

## 2.4 Peraturan Perundang-undangan

??? boleh ngequote sebanyak apa?

Menurut Undang-Undang Nomor 10 Tahun 2004 Pasal 7 Ayat 1, peraturan perundang-undangan adalah peraturan tertulis yang dibentuk oleh lembaga negara atau pejabat yang berwenang dan mengikat secara umum. Jenis dan hierarki peraturan perundang-undangan terdiri atas:

- Undang-Undang Dasar Negara Republik Indonesia Tahun 1945
- Undang-Undang/Peraturan Pemerintah Pengganti Undang-Undang
- Peraturan Pemerintah
- Peraturan Presiden
- Peraturan Daerah

Berikut adalah beberapa peranan perundang-undangan untuk Indonesia 1:

- Peraturan perundang-undangan merupakan kaidah hukum yang mudah dikenal (diidentifikasi), mudah diketemukan kembali, dan mudah ditelusuri. Sebagai kaidah hukum tertulis, bentuk, jenis,dan tempatnya jelas. Begitu pula pembuatnya.
- Peraturan perundang-undangan memberikan kepastian hukum yang lebih nyata karena kaidah-kaidahnya mudah diidentifikasi dan mudah diketemukan kembali.
- Struktur dan sistematika peraturan perundang-undangan lebih jelas sehingga memungkinkan untuk diperiksa kembali dan diuji baik segi-segi formal maupun materi muatannya.
- Pembentukan dan pengembangan peraturan perundang-undangan dapat direncanakan. Faktor ini sangat penting bagi negara-negara yang sedang membangun termasuk membangun sistem hukum baru yang sesuai dengan kebutuhan dan perkembangan masyarakat.

Menurut lampiran Undang-Undang Nomor 10 Tahun 2004, struktur peraturan perundang–undangan terdiri atas:

- Judul
- Pembukaan
- Batang Tubuh
- Penutup
- Penjelasan (jika diperlukan)
- Lampiran (jika diperlukan)

Bagian judul, pembukaan, dan penutup memuat metadata dari peraturan perundang-undangan tersebut seperti jenis, nomor, tahun pengundangan atau penetapan, nama, jabatan pembentuk, konsiderans, kasar Hukum, peraturan perundang-undangan. Konsiderans memuat uraian singkat mengenai pokok–pokok pikiran yang menjadi latar belakang dan alasan pembuatan peraturan perundang–undangan. Dasar Hukum memuat dasar kewenangan pembuatan peraturan perundang-undangan dan peraturan perundang–undangan yang memerintahkan pembuatan peraturan perundang–undangan tersebut. Oleh karena itu, kita dapat melihat keterkaitan antara peraturan perundang-undangan dengan melihat konsiderans dan dasar hukum dari peraturan tersebut.

Batang tubuh peraturan perundang-undangan memuat semua substansi peraturan perundang-undangan yang dirumuskan dalam pasal. Pengelompokkan materi peraturan perundang-undangan dapat disusun secara sistematis dalam buku, bab, bagian, dan paragraf atas dasar kesamaan materi. Pasal dapat dirinci ke dalam beberapa ayat. Pasal dan ayat dapat dibuat dalam bentuk kalimat maupun tabulasi rincian. Dalam bentuk tabulasi rincian, setiap rincian harus dapat dibaca sebagai satu rangkaian kesatuan dengan frase pembuka dan setiap rincian diawali dengan huruf (abjad) kecil dan diberi tanda baca titik. Suatu rincian dapat dibagi lagi ke unsur rincian yang lebih kecil.

Saat ini pembuatan dan pemanfaatan peraturan perundang-undangan masih dilakukan secara manual oleh manusia. Pembuatan peraturan perundang-undangan dilakukan dengan mengetikkan konten peraturan tersebut dengan format yang telah ditentukan. Metode pembuatan tersebut tidak masalah jika hanya akan dibaca oleh manusia karena manusia secara tidak sadar dapat melihat hirarki visual dari dokumen tersebut dan membuat data yang dilihatnya terstruktur di dalam otak. Metode ini membuat pemanfaatan peraturan perundang-undangan oleh mesin menjadi kurang efisien karena mesin perlu melakukan proses tambahan yaitu mengkonversi gambar menjadi data terstruktur.

## 2.5 Pemodelan European Legislation Identifier (ELI)

ELI merupakan sistem untuk membuat peraturan perundang-undangan tersedia secara daring dalam format terstandardisasi, sehingga dapat diakses dan digunakan oleh berbagai instansi. ELI bertujuan untuk untuk memfasilitasi akses, berbagi dan interkoneksi informasi hukum yang diterbitkan melalui nasional, Eropa dan sistem informasi hukum global. ELI dibangun berdasarkan persetujuan antara negara-negara EU. Spesifikasi yang terdapat pada ELI antara lain:

- URI untuk informasi peraturan perundang-undangan.
- Metadata yang mendeskripsikan informasi peraturan perundang-undangan.

# BAB 3
# PERANCANGAN

## 3.1 Perancangan URI

Setiap entitas yang berbeda perlu diberikan URI yang berbeda. Penggabungan KG sangat umum dilakukan karena secara struktur dapat dilakukan dengan mudah, dan dapat memperkuat data dari masing-masing KG. KG memiliki entitas-entitas dengan URI yang unik, oleh karena itu saat menggabungkan sebuah KG A dan sebuah KG B, entitas yang berbeda harus memiliki URI yang berbeda. Sebuah masalah dapat terjadi apabila URI tidak dirancang dengan benar. Sebagai contoh, apabila terdapat entitas dari KG A dengan URI `http://example.org/trump` dengan maksud Donald Trump mantan presiden Amerika Serikat, dan entitas dari KG B dengan URI yang sama dengan maksud permainan kartu Trump, saat kedua KG ini digabung akan menjadi informasi yang berkontradiksi.

Pada penelitian ini, semua URI menggunakan prefix `http://example.org/legal/` yang selanjutnya akan disebut *basePrefix*. Prefix ini dapat diganti dalam program agar dapat disesuaikan oleh pengguna program. Semua entitas dokumen dan komponennya menggunakan prefix URI `http://example.org/legal/peraturan` yang selanjutnya disebut *peraturanPrefix*, dan semua resource lainnya menggunakan `http://example.org/legal/ontology` yang selanjutnya disebut *ontoPrefix*.

## 3.2 Perancangan URI Class

URI Class adalah URI yang menjelaskan *class* dari entitas tersebut. Semua URI Class mengikuti pola `{ontoPrefix}/{className}` dimana `className` adalah nama *class* tersebut. Berikut adalah semua *class* komponen beserta URI dan deskripsinya.

| Nama Class | Deskripsi |
|---|---|
| `Peraturan` | Peraturan perundang-undangan |
| `DaftarBab` | Daftar satu atau lebih bab |
| `Bab` | Bab |
| `DaftarBagian` | Daftar satu atau lebih bagian |
| `Bagian` | Bagian |
| `DaftarParagraf` | Daftar satu atau lebih paragraf |
| `Paragraf` | Paragraf |

| Nama Class | Deskripsi |
|---|---|
| `DaftarPasal` | Daftar satu atau lebih pasal |
| `Pasal` | Pasal |
| `VersiPasal` | Versi dari sebuah pasal. Sebuah pasal dapat memiliki satu atau lebih versi dari hasil amandemen |
| `DaftarAyat` | Daftar satu atau lebih ayat |
| `Ayat` | Ayat |
| `DaftarHuruf` | Daftar satu atau lebih huruf atau nomor |
| `Huruf` | Huruf atau nomor |
| `Menimbang` | Hal yang ditimbang oleh suatu peraturan |
| `Mengingat` | Hal yang diingat oleh suatu peraturan |
| `Segmen` | Segmen teks. Dapat memiliki rujukan ke suatu komponen peraturan |

**Tabel 3.1:** Long table caption.

## 3.3 Perancangan URI Komponen Peraturan Perundang-undangan

URI peraturan perundang-undangan diawali oleh *peraturanPrefix*, kemudian diikuti oleh string yang mengidentifikasi peraturan tersebut. URI peraturan ini akan digunakan oleh entitas sebagai prefix yang selanjutnya akan disebut *docURI*. Berikut adalah jenis peraturan perundang-undangan yang diimplementasi pada penelitian ini beserta pola URI nya. Variabel akan dikurung dengan kurung kurawal `{}`.

| Jenis Peraturan | Pola URI | Penjelasan variabel |
|---|---|---|
| Undang-Undang Dasar 1945 | `/uud` | |
| Undang-Undang | `/uu/{thn}/{nmr}` | `thn`: tahun peraturan<br>`nmr`: nomor peraturan |
| Peraturan Pemerintah | `/pp/{thn}/{nmr}` | `thn`: tahun peraturan<br>`nmr`: nomor peraturan |
| Peraturan Daerah Provinsi DKI Jakarta | `/perda_provinsi_dki_jakarta/{thn}/{nmr}` | `thn`: tahun peraturan<br>`nmr`: nomor peraturan |
| Peraturan Gubernur DKI Jakarta | `/pergub_dki_jakarta/{thn}/{nmr}` | `thn`: tahun peraturan<br>`nmr`: nomor peraturan |

| Jenis Peraturan | Pola URI | Penjelasan variabel |
|---|---|---|
| Peraturan Walikota Malang | /perwali_malang/{thn}/{nmr} | thn: tahun peraturan<br>nmr: nomor peraturan |

**Tabel 3.2:** Long table caption.

Setiap komponen dokumen merupakan entitas, dan memiliki URI. URI sebuah komponen didahului oleh *docURI* dan semua komponen yang mengatasinya. Berikut adalah jenis komponen peraturan perundang-undangan beserta pola URI nya.

| Class Komponen | Pola URI | Penjelasan variabel |
|---|---|---|
| DaftarBab | {docURI}/bab | docURI: URI peraturan yang mengatasi |
| Bab | {daftarBabURI}/{no} | daftarBabURI: URI DaftarBab yang mengatasi<br>no: Nomor bab |
| DaftarBagian | {babURI}/bagian | babURI: URI Bab yang mengatasi |
| Bagian | {daftarBagianURI}/{no} | daftarBagianURI: URI DaftarBagian yang mengatasi<br>no: Nomor bagian |
| DaftarParagraf | {bagianURI}/paragraf | bagianURI: URI Bagian yang mengatasi |
| Paragraf | {daftarParagrafURI}/{no} | daftarParagrafURI: URI DaftarBagian yang mengatasi<br>no: Nomor paragraf |
| DaftarPasal | {parentURI}/daftarpasal | parentURI: URI Bab, Bagian, Paragraf yang mengatasi |
| Pasal | {docURI}/pasal/{no} | docURI: URI peraturan yang mengatasi<br>no: Nomor pasal |
| VersiPasal | {pasalURI}/versi/{date} | pasalURI: URI Pasal yang mengatasi<br>date: Tanggal disahkan |
| DaftarAyat | {pasalURI}/ayat | pasalURI: URI Pasal yang mengatasi |
| Ayat | {daftarAyatURI}/{no} | daftarAyatURI: URI DaftarAyat yang mengatasi<br>no: Nomor ayat |
| DaftarHuruf | {parentURI}/huruf | parentURI: URI Ayat, Huruf, VersiPasal, Menimbang, Mengingat, yang mengatasi |

| Class Komponen | Pola URI | Penjelasan variabel |
|---|---|---|
| Huruf | {daftarHurufURI}/{id} | daftarHurufURI: URI DaftarHuruf yang mengatasi<br>id: Nomor atau huruf |
| Menimbang | {docURI}/menimbang | docURI: URI peraturan yang mengatasi |
| Mengingat | {docURI}/mengingat | docURI: URI peraturan yang mengatasi |
| Segmen | {parentURI}/{name} | parentURI: URI Ayat, Huruf, DaftarHuruf, VersiPasal, Menimbang, Mengingat, yang mengatasi<br>name: Nama teks |

**Tabel 3.3:** Long table caption.

## 3.4 Perancangan URI Properti

Properti ditandai dengan URI yang memiliki pola {ontoPrefix}/{name} dimana name adalah nama dari properti tersebut. Properti menghubungkan subjek dan objek. Subjek adalah berupa URI komponen, sedangkan objek dapat berupa URI komponen, string, *number* (bilangan), atau *date* (tanggal).

| Nama | Subjek | Objek | Deskripsi |
|---|---|---|---|
| nomor | Bab, Bagian, Paragraf, Pasal, Ayat, Huruf | string, number | Nomor atau huruf _identifier_ dari komponen |
| teks | Ayat, VersiPasal, Segmen | string | teks dari komponen |
| bab | DaftarBab | Bab | Bab |
| bagian | DaftarBagian | Bagian | Bagian |
| paragraf | DaftarParagraf | Paragraf | Paragraf |
| pasal | DaftarPasal | Pasal | Pasal |
| ayat | DaftarAyat | Ayat | Ayat |
| huruf | DaftarHuruf | Huruf | Huruf |
| segmen | Mengigat, Menimbang, Paragraf, VersiPasal, Huruf, DaftarHuruf | Segmen | Segmen |

| Nama | Subjek | Objek | Deskripsi |
|---|---|---|---|
| daftarBab | Peraturan | DaftarBab | DaftarBab |
| daftarBagian | Bab | Daftarbagian | DaftarBagian |
| daftarParagraf | Bagian | DaftarParagraf | DaftarParagraf |
| daftarPasal | Peraturan,Bab, Paragraf, Bagian | DaftarPasal | DaftarPasal |
| daftarAyat | Pasal | DaftarAyat | DaftarAyat |
| daftarHuruf | VersiPasal, Ayat, Huruf, Menimbang, Mengingat | DaftarHuruf | DaftarHuruf |
| merujuk | Segmen | entitas apapun | Teks merujuk suatu entitas |
| mengubah | Huruf | VersiPasal | Pengubahan pasal |
| menghapus | Huruf | VersiPasal | Penghapusan pasal |
| menyisipkan | Huruf | VersiPasal | Penyisipan pasal |
| tanggal | VersiPasal | date | Tanggal |
| jenisVersi | VersiPasal | 'orisinal', 'penyisipan', 'pengubahan', 'penghapusan' | Jenis versi |
| versi | Pasal | VersiPasal | Versi suatu komponen |
| tentang | Peraturan | string | Peraturan tentang |
| menimbang | Peraturan | Menimbang | Peraturan menimbang |
| mengingat | Peraturan | Mengingat | Peraturan mengingat |
| disahkanPada | Peraturan | date | Peraturan disahkan pada tanggal |
| disahkanDi | Peraturan | string | Peraturan disahkan di lokasi |
| disahkanOleh | Peraturan | string | Peraturan disahkan oleh |
| jabatanPengesah | Peraturan | string | Jabatan pengesah peraturan |
| bagianDari | komponen apapun | komponen apapun | Komponen subjek adalah bagian dari komponen objek |

**Tabel 3.4:** Long table caption.

# BAB 4
# IMPLEMENTASI

## 4.1 OCR Ulang Berkas PDF

Dokumen peraturan perundang-undangan yang digunakan pada penelitian didapatkan dalam format berkas PDF. Penulis pada awalnya mencoba langsung mengekstraksi data dari PDF, tetapi penulis menemukan kesulitan. Salahsatu kesulitan yang penulis temui adalah terdapatnya salah pemindaian pada berkas PDF. Sebagai contoh, terdapat teks yang tertulis `Pasal` tetapi mengandung data `Pasai`. Selain itu, penulis juga menemukan bahwa setiap dokumen mengandung keslahan pemindaian yang bervariasi. Untuk suatu kasus, teks `(2)` selalu terdeteksi sebagai `(21` pada suatu dokumen, tetapi tidak pada dokumen lainnya. Pada kasus lainnya terdapat dokumen yang hampir tidak memiliki kesalahan pemindaian dan juga terdapat dokumen yang teksnya tidak dapat dibaca samasekali.

Walaupun dokumen-dokumen tersebut dipelihara oleh satu lembaga pada satu situs web, yaitu oleh Dewan Perwakilan Rakyat pada situs web dpr.go.id/jdih, masing-masing dokumen dibuat kedalam berkas PDF dengan cara yang berbeda-beda. Penulis tidak dapat mengetahui secara pasti metode apa yang digunakan, tetapi dari metadata yang didapatkan dari berkas PDF, penulis dapat membuat beberapa dugaan. Pada berkas PDF, terdapat metadata dengan nama **Creator**, dimana pada dokumen-dokumen peraturan Perudang-udangan yang didapatkan, tercantum nama-nama alat pencetak atau merk dari pencetak tersebut. Dari informasi tersebut penulis menduga bahwa terdapat sebagian peraturan perundang-undangan yang dibuat menjadi PDF dengan mencetaknya menjadi kertas terlebih dahulu kemudian di pindai oleh pemindai dan sebagian lainnya dikonversi langsung dari berkas *.docx*. Data yang dipindai adalah berupa gambar, artinya teks yang terdapat pada berkas PDF adalah hasil OCR (*optical character recognition*) oleh pemindai tersebut. Berikut adalah salahsatu contoh dokumen beserta data yang tercantum sebagai **Creator** dan contoh kesalahan pemindaiannya yang banyak terjadi.

| Dokumen | __Creator__ | Kesalahan pemindaian |
|---|---|---|
| UU Nomor 13 Tahun 2003 | ScanSoft PDF Create! 4 | - (tidak ada) |
| UU Nomor 6 Tahun 2018 | Canon | ‘(2)‘ selalu dipindai ‘(21‘ |
| PP Nomor 34 Tahun 2021 | Fuji Xerox B9100 | data teks tidak terbaca |

Pemindaian berkas PDF dengan metode yang berbeda-beda memberikan kualitas hasil pemindaian yang berbeda-beda dan kesalahan pemindaian yang tidak konsisten. Un-

tuk menyelesaikan masalah ini, penulis melakukan OCR ulang terhadap semua dokumen yang akan di konversi. Dengan melakukan OCR ulang menggunakan satu metode yang sama, penulis tidak hanya berhasil mendapatkan data hasil pemindaian berkas PDF dengan kualitas pemindaian yang konsisten untuk semua dokumen. Penulis memilih menggunakan Tesseract OCR 4 sebagai metode OCR karena sifatnya *open source* dan mendukung Bahasa Indonesia sebagai bahasa yang dipindai.

Untuk melakukan OCR ulang perlu dilakukan instalasi program yang dibutuhkan yang dapat dilakukan dengan *command* berikut pada sistem operasi Ubuntu 20.04. `ocrmypdf` merupakan *wrapper* untuk Tesseract OCR dan `tesseract-ocr-ind` adalah program tambahan untuk dapat melakukan OCR menjadi teks Bahasa Indonesia.

```
apt update apt install ocrmypdf tesseract-ocr-ind -y
```

Setelah program diinstall, OCR dapat dilakukan dengan menjalankan *command* dibawah. Pada contoh dibawah dilakukan konversi `UU-2021-34.pdf` yang sudah ada menjadi `UU-2021-34_OCR.pdf` hasil OCR. Opsi `-force-ocr` diperlukan untuk melakukan OCR ulang pada berkas PDF yang sudah memiliki teks hasil OCR untuk di *overwrite*. Opsi `-jobs 4` menjalankan *command* dengan 4 *core* CPU, bilangan pada opsi ini dapat diganti sesuai kebutuhan. Opsi `-tesseract-config tesseract-config.cfg` menjalankan *command* dengan konfigurasi yang diberikan.

```
ocrmypdf -l ind --force-ocr --jobs 4 --tesseract-config tesseract-config.cfg
↪  UU-2021-34.pdf UU-2021-34_OCR.pdf
```

Konfigurasi Tesseract OCR yang digunakan adalah `tessedit_pageseg_mode 4` seperti yang terlihat pada isi file konfigurasi dibawah. `tessedit_pageseg_mode 4` digunakan agar Tesseract OCR mendeteksi teks sebagai *block*, bukan *word* atau *character*.

```
# tesseract-config.cfg
tessedit_pageseg_mode 4
```

## 4.2 Ekstraksi Berkas PDF menjadi Data *Span*

Agar dapat diproses oleh program, berkas PDF perlu diolah menjadi data berupa daftar teks dan posisinya yang selanjutnya akan disebut *span*. Sebuah *span* mengandung satu baris teks. Berikut adalah data yang dimiliki oleh sebuah *span*:

- `str`: teks yand dikandung
- `xL`: koordinat titik terkiri dari *span*

- `xR`: koordinat titik terkanan dari *span*
- `y`: koordinat titik teratas dari *span*
- `pageNum`: nomor halaman
- `id`: *identifier* unik untuk setiap span

Berikut adalah contoh gambaran berkas PDF dan hasil ekstraksinya menjadi daftar *span*.

**UNDANG-UNDANG REPUBLIK INDONESIA**

**NOMOR 13 TAHUN 2003**

**TENTANG**

**KETENAGAKERJAAN**

**Gambar 4.1:** Gambar dokumen PDF yang akan diekstraksi

akan dipindai menjadi

```
- xL: 197.52
  xR: 442.79990000000004
  'y': 234.48000000000002
  str: UNDANG-UNDANG REPUBLIK INDONESIA
  pageNum: 1
  id: 0
- xL: 252.72
  xR: 387.6002
  'y': 255.12
  str: NOMOR 13 TAHUN 2003
  pageNum: 1
  id: 1
- xL: 291.12
  xR: 349.68048
  'y': 275.76
  str: TENTANG
  pageNum: 1
  id: 2
- xL: 257.28
  xR: 383.28015999999997
  'y': 296.4
  str: KETENAGAKERJAAN
  pageNum: 1
  id: 3
```

## 4.3 Membersihkan *Noise* dari Halaman

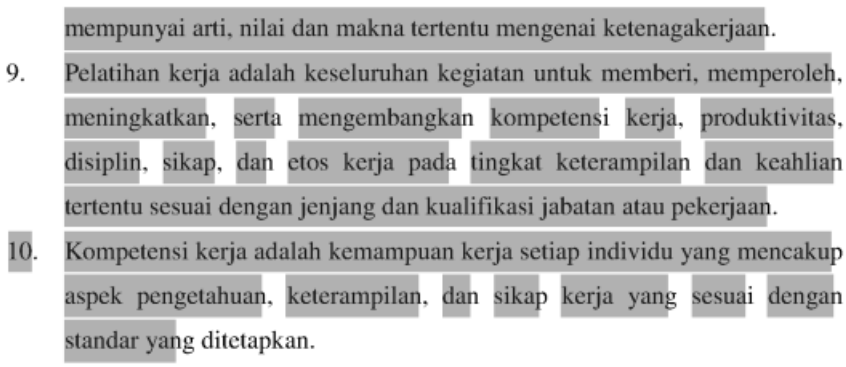
Tidak jarang dokumen peraturan Perudang-undangan mengandung data *noise* yang tidak ingin kita ekstraksi seperti *header* dan *footer*. Pada penelitian ini, penulis menggunakan dokumen dari sumber yang sama sehingga memiliki format yang sama, dan juga posisi *header* dan *footer* yang hampir sama pada setiap dokumen.

**Gambar 4.2:** Contoh header

Gambar diatas adalah contoh *header* yang terdapat pada setiap halaman. Dapat dilihat bahwa *header* selalu terdiri dari teks "PRESIDEN REPUBLIK INDONESIA" dan diikuti oleh nomor halaman, dan selalu terletak di posisi yang hampir sama. Oleh karena itu, penulis memeriksa teks menggunakan *regex* dan posisi dari setiap *span*, kemudian menghapus *span* tersebut jika terdeteksi sebagai header.

## 4.4 Penggabungan Data Berkas PDF Asli dan Hasil OCR Ulang

Pada proses ekstraksi berkas PDF menjadi daftar *span*, penulis menemukan satu masalah yaitu data hasil OCR tidak konsisten dalam memindai angka. Seperti yang dapat dilihat pada gambar dibawah, angka 10 berhasil dipindai tetapi angka 9 tidak berhasil. Untuk kasus dibawah, angka hanya tidak terpindai pada hasil OCR ulang, tetapi terpindai dengan benar pada berkas PDF aslinya. Untuk menyelesaikan masalah tersebut, penulis melakukan ekstraksi data pada berkas PDF aslinya, kemudian menggabungkannya dengan data hasil pemindaian untuk melengkapi bagian yang tidak terpindai.




**Gambar 4.3:** Contoh nomor tidak terpindai

Penulis hanya menggabungkan data dua berkas PDF untuk kasus angka seperti yang disebutkan diatas. Hal tersebut dikarenakan pada umumnya kita dapat mentolerir kesalahan pemindaian pada teks, tetapi karena kegagalan pemindaian pada angka tersebut akan mempengaruhi struktur dokumen, penulis memutuskan solusi diatas. Sebagai contoh, jika nomor 8 berhasil dipindai dan nomor 9 tidak berhasil, maka struktur akhir yang dihasilkan akan mengandung semua isi nomor 9 didalam nomor 8, yang mana seharusnya adalah nomor yang terpisah. Berikut berturut-turut adalah contoh data yang dihasilkan jika nomor 9 tidak berhasil terpindai dan jika berhasil terpindai.

Jika nomor 9 tidak berhasil terpindai:

```
- type: point
  key: 8
  text: >-
    Informasi ketenagakerjaan adalah gabungan, rangkaian, dan analisis data yang
    berbentuk angka yang telah diolah, naskah dan dokumen yang mempunyai arti,
    nilai dan makna tertentu mengenai ketenagakerjaan.
    9. Pelatihan kerja adalah keseluruhan kegiatan untuk memberi, memperoleh,
    meningkatkan, serta mengembangkan kompetensi kerja, produktivitas, disiplin,
    sikap, dan etos kerja pada tingkat keterampilan dan keahlian tertentu sesuai
    dengan jenjang dan kualifikasi jabatan atau pekerjaan.
```

Jika nomor 9 berhasil terpindai:

```
- type: point
  key: 8
  text: >-
    Informasi ketenagakerjaan adalah gabungan, rangkaian, dan analisis data yang
    berbentuk angka yang telah diolah, naskah dan dokumen yang mempunyai arti,
    nilai dan makna tertentu mengenai ketenagakerjaan.
- type: point
  key: 9
  text: >-
    Pelatihan kerja adalah keseluruhan kegiatan untuk memberi, memperoleh,
    meningkatkan, serta mengembangkan kompetensi kerja, produktivitas, disiplin,
    sikap, dan etos kerja pada tingkat keterampilan dan keahlian tertentu sesuai
    dengan jenjang dan kualifikasi jabatan atau pekerjaan.
```

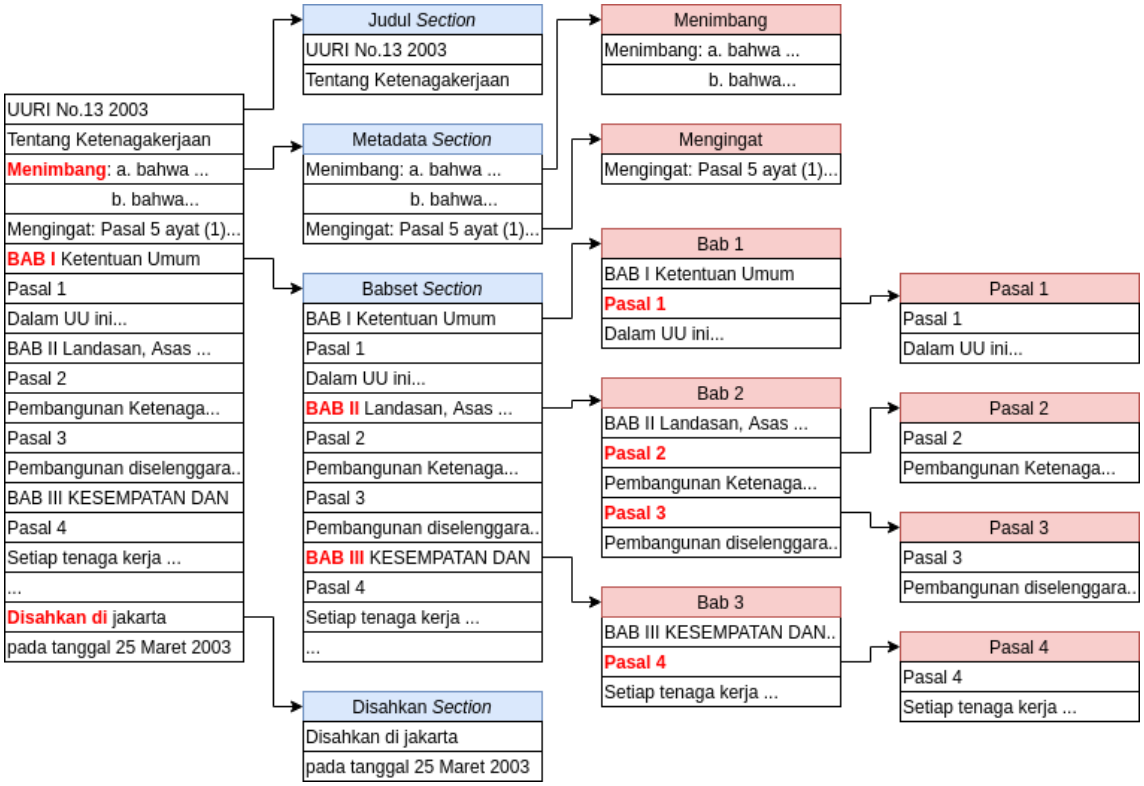
## 4.5 Klasifikasi Dokumen

Pa

## 4.6 Pengelompokan *span* menjadi Komponen

Teks yang membentuk suatu komponen terdiri dari satu atau lebih *span*, sehingga daftar *span* yang didapatkan dari hasil ekstraksi berkas PDF harus dikelompokkan sehingga setiap kelompok merepresentasikan *span* yang terdapat pada suatu komponen. Pengelompokan dilakukan dengan mendeteksi *span* awal dan *span* akhir dari sebuah komponen. Karena daftar *span* hasil ekstraksi bersifat 1 dimensi, hal ini bisa dilakukan dengan melakukan iterasi pada setiap *span*, kemudian menandai awal atau akhir dari sebuah kelompok apabila memenuhi suatu pola. Setelah ditandai, *span* akan dikelompokkan sebagai daftar *span* dari suatu komponen.

Pengelompokan tidak langsund dilakukan dari daftar *list* menjadi daftar komponen, melainkan dibuat pengelompokan *section* untuk pembagian dokumen secara garis besar, kemudian baru dikelompokkan sebagai komponen dari masing-masing *section* tersebut. Pada gambar dibawah *section* ditandai dengan warna biru dan komponen ditandai dengan warna merah. Dapat dilihat bahwa daftar *span* data awal dikelompokkan menjadi beberapa *section* yaitu `judul`, `metadata`, `babset`, dan `disahkan`. Kemudian *span* pada section `babset` dikelompokkan menjadi komponen bab yaitu `Bab 1`, `Bab 2`, dan `Bab 3`. Kemudian *span* pada setiap komponen bab dikelompokkan menjadi komponen pasal. Pengelompokan bertingkat ini dilakukan agar fungsi untuk mendeteksi batas antara komponen tidak menjadi rumit.




**Gambar 4.4:** Ilustrasi Ekstraksi Daftar Span menjadi Data Struktur Komponen

Deteksi batas antara komponen atau *section* dilakukan dengan melihat data pada *span* seperti `str`, `xL`, `xR`, dan `y`. Pada gambar diatas, pola yang terdeteksi sebagai batas ditandai dengan huruf merah. Sebagai contoh, pada ekstraki paling kiri, dari daftar *span* asli menjadi *section*, dilakukan iterasi *span* dari atas dan akan dimasukkan ke dalam *section* `judul` sampai menemukan *section* yang diawali dengan kata **Menimbang**. Kemudian setiap *span* setelah span **Menimbang** tersebut akan dikelompokkan menjadi *section* `metadata` sampai menemukan section yang diawali kata **BAB I**.

### 4.6.1 Komponen Terurut

Beberapa komponen seperti `bab`, `bagian`, `paragraf`, `pasal`, `angka`, dan `huruf` memiliki sifat terurut. Dalam hal ini, yang termasuk terurut adalah:

- Komponen yang mengawali suatu komponen terurut akan selalu sama.
- Hanya terdapat satu komponen yang dapat mengikuti komponen terurut.

Contoh dari sifat pertama adalah `bab`, `bagian`, `paragraf`, `pasal`, `angka` pasti dimulai dari komponen nomor 1 seperti `bab 1` dan `pasal 1`, dan `huruf` pasti dimulai dari komponen `huruf a`. Contoh dari sifat kedua adalah hanya `pasal 2` yang dapat mengikuti

pasal 1 dan hanya huruf b yang dapat mengikuti huruf a. Dalam implementasi, penulis memastikan sifat-sifat ini dipatuhi, sehingga sebagai contoh apabila sebuah dokumen terdeteksi mengandung bab 12 maka akan dijamin mengandung semua bab 1 sampai bab 11 dan apabila terdeteksi mengandung pasal 196 maka akan dijamin mengandung semua pasal dari pasal 1 sampai pasal 195.

### 4.6.2 Komponen Amandemen

Mengekstraksi *span* pada bagian amandemen merupakan salahsatu tantangan pada penelitian ini. Dibawah adalah gambar amandemen pada Pasal 17 UU Nomor 11 Tahun 2020 yang dilakukan terhadap Pasal 1 UU Nomor 26 Tahun 2007. Dapat dilihat bahwa kata Pasal 17 dan Pasal 1 yang digunakan sebagai pembatas antara komponen memiliki pola teks dan posisi yang hampir sama. Hal ini membuat pembedaan antara "Pasal biasa" dan "Pasal peng-amandemen" menjadi sulit dilakukan.

Pasal 17

Beberapa ketentuan dalam Undang-Undang Nomor 26 Tahun 2007 tentang Penataan Ruang (Lembaran Negara Republik Indonesia Tahun 2007 Nomor 68, Tambahan Lembaran Negara Republik Indonesia Nomor 4725) diubah sebagai berikut:

1. Ketentuan Pasal 1 angka 7, angka 8, dan angka 32 diubah sehingga Pasal 1 berbunyi sebagai berikut:

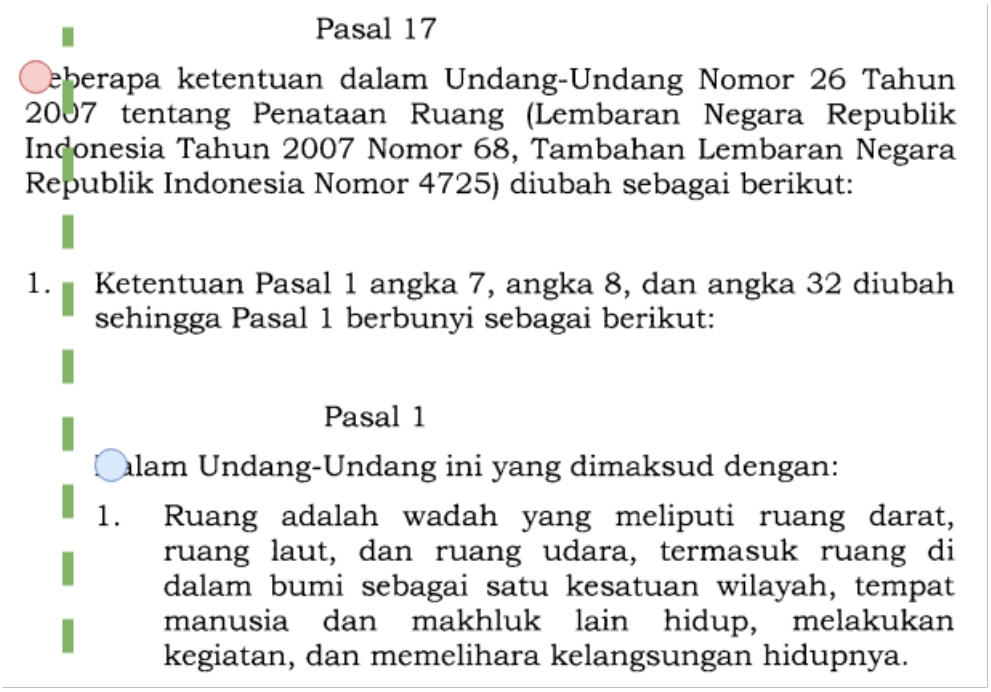
Pasal 1

Dalam Undang-Undang ini yang dimaksud dengan:

1. Ruang adalah wadah yang meliputi ruang darat, ruang laut, dan ruang udara, termasuk ruang di dalam bumi sebagai satu kesatuan wilayah, tempat manusia dan makhluk lain hidup, melakukan kegiatan, dan memelihara kelangsungan hidupnya.

**Gambar 4.5:** Contoh Amendemen pada UU Nomor 11 Tahun 2020

Untuk menyelesaikan masalah ini, penulis menggunakan koordinat xL *span* tepat setelah *span* yang bertuliskan "Pasal X". Pada contoh deteksi kompoonen amandemen di bawah, xL dari teks setelah pasal peng-amandemen (pasal biasa) ditandai dengan lingkaran merah, xL dari pasal yang diamandemen ditandai dengan lingkaran biru, dan garis hijau merepresentasikan batas pemeda xL antara pasal biasa dan pasal yang diamandemen. Pembedaan pasal biasa dan pasal amandemen hanya dilakukan jika dokumen memiliki komponen berupa amandemen.



Pasal 17

Beberapa ketentuan dalam Undang-Undang Nomor 26 Tahun 2007 tentang Penataan Ruang (Lembaran Negara Republik Indonesia Tahun 2007 Nomor 68, Tambahan Lembaran Negara Republik Indonesia Nomor 4725) diubah sebagai berikut:

1. Ketentuan Pasal 1 angka 7, angka 8, dan angka 32 diubah sehingga Pasal 1 berbunyi sebagai berikut:

Pasal 1

Dalam Undang-Undang ini yang dimaksud dengan:

1. Ruang adalah wadah yang meliputi ruang darat, ruang laut, dan ruang udara, termasuk ruang di dalam bumi sebagai satu kesatuan wilayah, tempat manusia dan makhluk lain hidup, melakukan kegiatan, dan memelihara kelangsungan hidupnya.

**Gambar 4.6:** Deteksi Komponen Amandemen

## 4.7 Deteksi Sitasi

Setelah semua *span* dikelompokan menjadi suatu komponen, dilakukan deteksi sitasi pada teks pada setiap komponen untuk mengetahui apakah komponen tersebut menyebut komponen atau dokumen peraturan perundang-undangan lainnya. Berikut adalah daftar pola sitasi yang berhasil dideteksi pada penelitian ini.

| Pola | Contoh teks | URI Terdeteksi |
|---|---|---|
| UUD 1945 | Undang Undang Dasar Negara Republik Indonesia Tahun 1945 | uud |
| Undang-Undang Nomor {x} Tahun {y} | Undang-Undang Nomor 26 Tahun 2007 | `uu/2007/26` |
| ayat ({x}) | ayat (1) | `uu/2003/13/pasal/169/ayat/1` |
| Pasal {x} | Pasal 156 | `uu/2003/13/pasal/156` |
| Pasal {x} ayat ({y}) | Pasal 156 ayat (2) | `uu/2003/13/pasal/156/ayat/2` |

| Pola | Contoh teks | URI Terdeteksi |
|---|---|---|
| huruf {x}, {y}, ... dan {z} | huruf a, b, c, d, dan e | uu/2003/13/menimbang/point/a<br>uu/2003/13/menimbang/point/b<br>uu/2003/13/menimbang/point/c<br>uu/2003/13/menimbang/point/d<br>uu/2003/13/menimbang/point/e |
| huruf {x} dan {y} | huruf a dan b | uu/2003/13/pasal/1/point/5/point/a<br>uu/2003/13/pasal/1/point/5/point/b |

**Tabel 4.1:** Long table caption.

Data sitasi mengandung data sebagai berikut:

- `start`: *index* karakter awal sitasi
- end: *index* karakter akhir sitasi
- `uri`: URI dokumen dan

Berikut adalah contoh teks pada Undang-Undang Nomor 13 Tahun 2003 Pasal 158 Ayat (4) beserta sitasi yang terdeteksi.

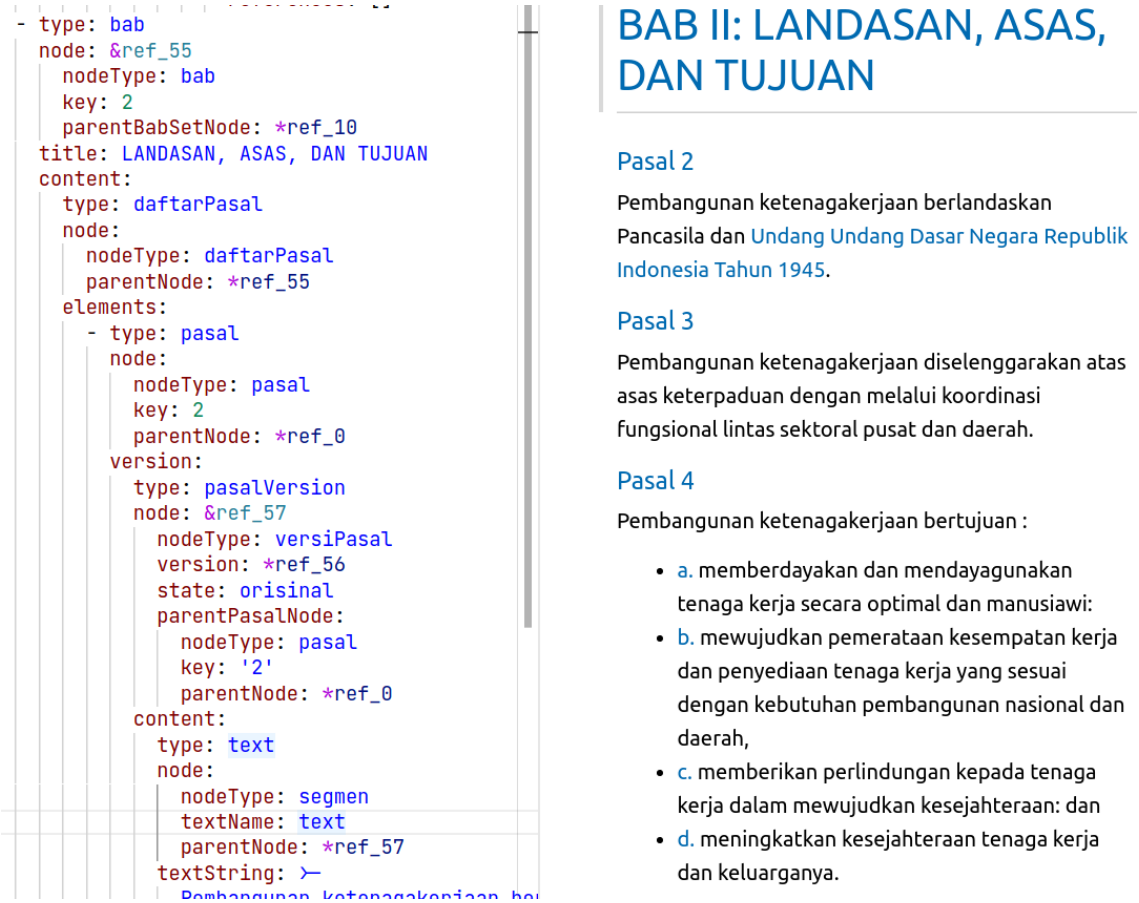
```
textString: >-
  Pekerja/buruh yang diputus hubungan kerjanya berdasarkan alasan sebagaimana
  dimaksud dalam ayat (1), dapat memperoleh uang penggantian hak sebagaimana
  dimaksud dalam Pasal 156 ayat (4).
references:
  - start: 91
    end: 99
    uri: /uu/2003/13/pasal/158/ayat/1
  - start: 166
    end: 184
    uri: /uu/2003/13/pasal/156/ayat/4
```

Untuk melakukan deteksi, teks akan melalui fungsi yang masing-masing memiliki pola untuk dideteksi. Oleh karena itu, untuk *substring* yang beririsan dapat memiliki lebih dari dua sitasi yang terdeteksi. Pada kasus tersebut akan diambil sitasi dengan *substring* terpanjang. Sebagai contoh, teks “Pasal 156 Ayat (4)” pada UU No.13 Tahun 2003 Pasal 158 Ayat (4) akan terdeteksi sebagai 3 URI sitasi sebagai berikut.

| Teks | URI |
|---|---|
| Pasal 156 Ayat (4) | `/uu/2003/13/pasal/156/ayat/4` |
| Pasal 156 | `/uu/2003/13/pasal/156` |
| Ayat (4) | `/uu/2003/13/pasal/158/ayat/4` |

Pada kasus ini, yang akan diambil adalah teks “Pasal 156 ayat (4)” karena merupakan teks terpanjang.

## 4.8 Visualisasi dengan Markdown




**Gambar 4.7:** Perbandingan visualisasi dengan yaml (kiri) dan markdown (kanan)

## 4.9 Data to triple

## 4.10 triple to ttl

## 4.11 Query

Fuseki

# BAB 5
# EVALUASI DAN ANALISIS HASIL

## 5.1 Aplikasi *Knowledge Graph* Peraturan Perundang-undangan

Dibuat chatbot untuk mensimulasikan pengaplikasian KG. Chatbot akan memproses input yand diberikan oleh user, kemudian meng-generate SPARQL query. Chatbot akan menampilkan hasil dari query tersebut.

## 5.2 Evaluasi Hasil Konversi

Evaluasi akan dilakukan terhadap semua dokumen yang terdapat pada peraturan.go.id. Selanjutnya dokumen-dokumen ini akan disebut dokument tes.

## 5.3 Evaluasi Manual

Evaluasi kualitatif dilakukan terhadap sebagian dokumen yang didapatkan dari hasil sampling dari dokumen tes. Seorang penilai akan melihat markdown dari setiap dokumen hasil *sampling*, masing-masing dokumen akan diberikan skor 0 sampai 100. Kualitas konversi dapat dilihat dari rata-rata skor tersebut.

## 5.4 Evaluasi Otomatis

Evaluasi dilakukan dengan terlebih dahulu mendefinisikan metadata yang pasti terdapat pada sebuah dokumen (misal, tempat dokumen disahkan), kemudian menghitung seberapa banyak metadata yang berhasil diekstrak. Metadata hasil ekstraksi yang akan dihitung tidak perlu sesuai dengan yang terdapat di dalam dokumen.

# BAB 6
# PENUTUP

Pada bab ini, Penulis akan memaparkan kesimpulan penelitian dan saran untuk penelitian berikutnya.

## 6.1 Kesimpulan

Berikut ini adalah kesimpulan terkait pekerjaan yang dilakukan dalam penelitian ini:

1. **Poin pertama**
   Penjelasan poin pertama.
2. **Poin kedua**
   Penjelasan poin kedua.

Tulis kalimat penutup di sini.

## 6.2 Saran

Berdasarkan hasil penelitian ini, berikut ini adalah saran untuk pengembangan penelitian berikutnya:

1. Saran 1.
2. Saran 2.

# DAFTAR REFERENSI

Paulheim, H. (2016). Knowledge graph refinement: A survey of approaches and evaluation methods. *Semantic Web*, 1-20.

Seth, Y. (2019, Oct). *Introduction to question answering over knowledge graphs.* Retrieved from https://yashuseth.blog/2019/10/08/introduction-question-answering-knowledge-graphs-kgqa/

# LAMPIRAN

# LAMPIRAN 1: CHANGELOG

lorem ipsum